SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Analytical Problem Solving
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/nonrestoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	Deepan Kumar P	**Reg. No.:**	192312498
Section / Batch:	Final year ECE	**Date:**	03/08/2026

Instructions to Students

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A digital computer performs signed integer arithmetic using 8-bit 2's complement representation. Consider the numbers −75 and +38. Analyze how the processor performs both addition and subtraction by converting the operands to binary, executing the operations step-by-step, detecting overflow (if any), and interpreting the final results in both binary and decimal forms.
2. A high-performance processor employs an 8-bit carry look-ahead adder (CLA) to enhance arithmetic speed. Given two binary operands, analyse how the generate and propagate signals are produced, how carry bits are calculated simultaneously, and compare the execution time and hardware complexity of both adders. Explain why CLA is widely used in modern processors.
3. A digital processing unit executes signed multiplication using Booth's algorithm. Multiply +13 and −6, and analyze the algorithm step-by-step by showing the contents of the accumulator, multiplier, multiplicand, and Booth bit after each iteration. Explain the purpose of arithmetic right shifts and discuss how Booth's algorithm reduces the number of addition and subtraction operations.
4. A binary arithmetic unit supports both restoring and non-restoring division methods for signed integer operations. Analyze the division of the binary numbers 27 ÷ 5 using each algorithm by showing the intermediate remainder values, quotient formation, and register contents after every iteration. Compare the two methods based on execution steps, computational complexity, and hardware efficiency, and justify which algorithm is more suitable for high-speed processors.
5. A floating-point processing unit represents real numbers using the IEEE 754 standard. Analyze the representation of the decimal number −42.375 in both single-precision (32-bit) and doubleprecision (64-bit) formats by identifying the sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between −42.375 and +15.125, including exponent alignment, mantissa operations, normalization, rounding, and the impact of precision on the final result.

Code of Conduct

I Deepan Kumar P (192312498) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Deepan Kumar P

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 20**	**/ 25**	**/ 20**	**/ 15**	**/ 100**

Course Coordinator Faculty Incharge

1. Problem Understanding

The processor uses 8-bit Two's Complement representation to perform signed arithmetic.

Given

A = −75

B = +38

Operations

Addition

(-75) + (+38)

Subtraction

(-75) − (+38)

Method Selection

Use

8-bit Two's Complement Representation

Binary Addition

Two's Complement for subtraction

Overflow Detection Rule

Step 1 : Convert Numbers into Binary

(+75)

Decimal

75

Binary

75 = 64 + 8 + 2 +1

Binary

01001011

(-75)

Take Two's Complement

Positive

01001011

Invert

10110100

Add 1

10110101

Therefore

−75 = 10110101

(+38)

38

Binary

38 = 32 +4 +2

Binary

00100110

Therefore

+38 = 00100110

Operation 1

(-75) + (+38)

```
    10110101
+   00100110
-----------
    11011011
```

Convert Result to Decimal
Since MSB = 1
Number is negative.
Take Two's Complement
11011011
Invert
00100100
Add 1
00100101
Decimal
37
Therefore
Result
−37

Operation 2

(-75) − (+38)

Find Two's Complement of +38
+38
00100110
Invert
11011001
Add 1
11011010
Thus
−38
11011010
```
    10110101
+   11011010
----------------
1 10001111
```
Convert to Decimal
Take Two's Complement
10001111
Invert
01110000
Add 1
01110001
Decimal
113
Therefore
Result
−113

2. Method Selection

The Carry Look-Ahead Adder uses the following equations:
Generate signal:

$$G_i = A_i \cdot B_i$$

Propagate signal:

$$P_i = A_i \oplus B_i$$

Sum:

$$S_i = P_i \oplus C_i$$

Carry equations:

$$C_1 = G_0 + P_0C_0$$
$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$
$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

Similarly,

$$C_4, C_5, C_6, C_7, C_8$$

Solution Process

Step 1

Apply the Generate equation

$$G_i = A_iB_i$$

to every bit.

Step 2

Apply the Propagate equation

$$P_i = A_i \oplus B_i$$

to every bit.

Step 3

Using the above values, calculate all carry bits simultaneously using the carry equations. Unlike Ripple Carry Adder, the CLA does **not wait** for the previous carry.

Step 4

Calculate the sum bits

$$S_i = P_i \oplus C_i$$

to obtain the final binary result.

Comparison of CLA and Ripple Carry Adder

Parameter	Ripple Carry Adder	Carry Look-Ahead Adder
Carry Generation	Sequential	Parallel
Speed	Slow	Very Fast
Propagation Delay	High	Low
Hardware Complexity	Low	High
Number of Gates	Less	More
Processor Performance	Moderate	Excellent

---

2. Multiply +13 × −6 using Booth's Algorithm.

Method Selection

Use 8-bit Booth's Algorithm.

Rules:

- $Q_0Q_{-1} = 00 \rightarrow$ No operation
- $Q_0Q_{-1} = 01 \rightarrow A = A + M$
- $Q_0Q_{-1} = 10 \rightarrow A = A - M$
- $Q_0Q_{-1} = 11 \rightarrow$ No operation

After every operation perform Arithmetic Right Shift (ARS).

Iteration	Q_0Q_{-1}	Operation	A (after operation)	After Arithmetic Right Shift (A Q Q_{-1})
Initial	-	-	00000000	00000000 11111010 0
1	00	No operation	00000000	00000000 01111101 0
2	10	A = A − M	11110011	11111001 10111110 1
3	01	A = A + M	00000110	00000011 01011111 0
4	10	A = A − M	11110110	11111011 00101111 1
5	11	No operation	11111011	11111101 10010111 1
6	11	No operation	11111101	11111110 11001011 1
7	11	No operation	11111110	11111111 01100101 1
8	11	No operation	11111111	11111111 10110010 1

1111111110110010 answer

+13 × (−6) = −78

---

4. Problem Understanding

The processor performs binary division using:

- **Restoring Division Algorithm**
- **Non-Restoring Division Algorithm**

Given:

Dividend = 27

Divisor = 5

Dividend (Q) = 11011

Divisor (M) = 00101

Iteration	Shift Left (AQ)	A = A − M	A < 0 ?	Action	Quotient Bit
Initial	00000 11011	—	—	—	—
1	00001 10110	11100	Yes	Restore A	0
2	00011 01100	11110	Yes	Restore A	0
3	00110 11000	00001	No	Keep A	1
4	00011 10010	11110	Yes	Restore A	0
5	00111 00100	00010	No	Keep A	1

Quotient = 00101 = 5

Remainder = 00010 = 2

27 ÷ 5

Quotient = 5

Remainder = 2

Non-Restoring Division Algorithm

A = 00000

Q = 11011

M = 00101

Iteration	Shift Left	Operation	A	Quotient Bit
Initial	—	—	00000	—
1	Shift	A=A−M	11100	0
2	Shift	A=A+M	00000	1
3	Shift	A=A−M	11100	0
4	Shift	A=A+M	00010	1
5	Shift	A=A−M	00010	1

Quotient = 00101 = 5

Remainder = 00010 = 2

---

5. The Floating Point Unit (FPU) stores real numbers using the IEEE 754 standard.
Given:

- **Number 1 = −42.375**
- **Number 2 = +15.125**

Step 1: Convert −42.375 to Binary

$$42 = 101010_2$$

Fraction Part

0.375

Multiply by 2:

Calculation	Bit
0.375 ×2 =0.75	0
0.75 ×2 =1.5	1
0.5 ×2 =1.0	1

$$0.375 = .011_2$$

$$42.375 = 101010.011_2$$

101010.011

↓

$$1.01010011 \times 2^5$$